

besmeAllah

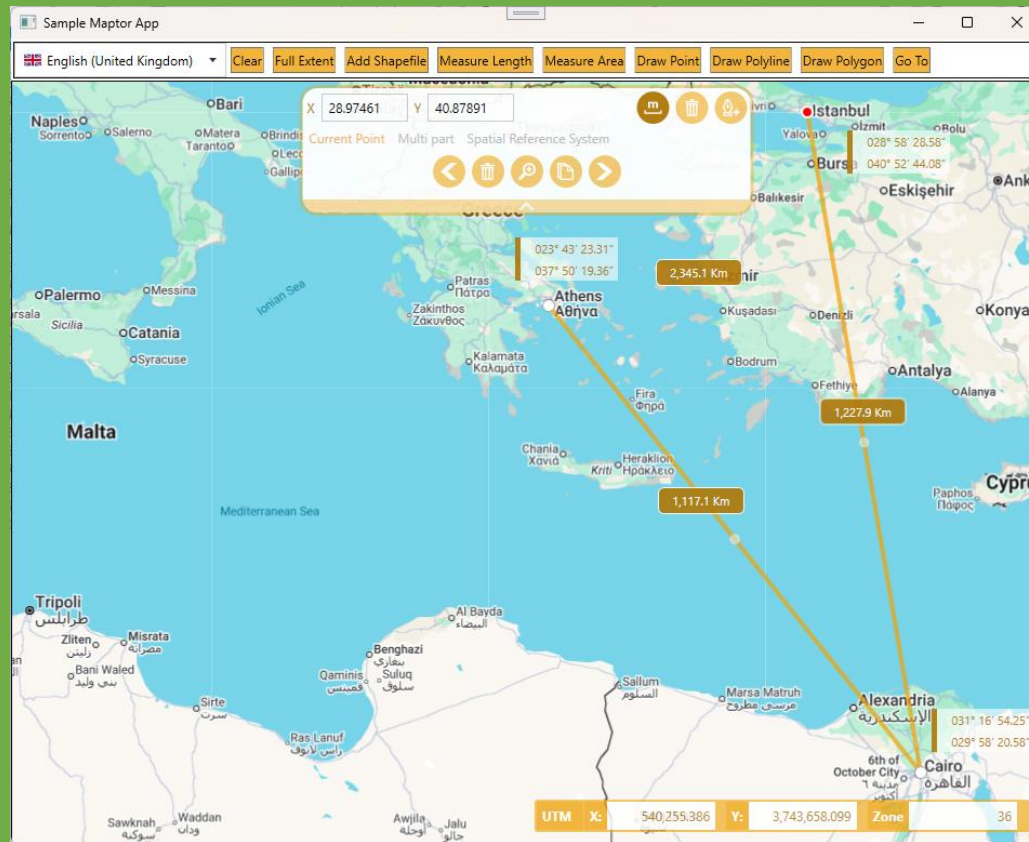
Build a Map-Based Application in 100 Lines of Code with Maptor

Maptor tutorial series

Hossein Narimani Rad @ **maptor**

swipe →

Final Output Preview



What You'll Learn

By the end of this tutorial, you'll know how to build a fully functional map-based application using **just 100 lines of code** and the built-in features of **Maptor**. Here's what your app will include:

- No external dependencies except **.NET 8**
- Smooth map interactions (**pan, zoom, etc.**)
- Google Maps as the base map
- Display of the current mouse position
- Support for adding **Shapefiles**
- Built-in tools to measure **length** and **area** on the map

Prerequisites

Before you start, make sure you have:

- [.NET 8 SDK](#) installed
- Visual Studio 2019 or later

Project Setup

1. Create a new **WPF project** Visual Studio targeting **.NET 8** (or higher).
2. Install the required Maptor NuGet package.



```
dotnet add package IRI.Maptor.Jab.Controls
```

3. Reference the Maptor's resource files in **App.xaml**



```
<ResourceDictionary>
  <ResourceDictionary.MergedDictionaries>
    <ResourceDictionary Source="pack://application:,,,/IRI.Maptor.Jab.Common;component/Assets/Styles/ButtonStyles.xaml"/>
    <ResourceDictionary Source="pack://application:,,,/IRI.Maptor.Jab.Common;component/Assets/Shapes/Others.xaml"/>
    <ResourceDictionary Source="pack://application:,,,/IRI.Maptor.Jab.Common;component/Assets/IRI.Converters.xaml"/>
    <ResourceDictionary Source="pack://application:,,,/IRI.Maptor.Jab.Common;component/Assets/IRI.Fonts.xaml"/>
    <ResourceDictionary Source="pack://application:,,,/IRI.Maptor.Jab.Common;component/Assets/IRI.Colors.xaml"/>
  </ResourceDictionary.MergedDictionaries>
</ResourceDictionary>
```

Prepare the map in XAML

1. Define the **Maptor namespace** in your `MainWindow.xaml` (or the desired window).



```
<Window ...  
    xmlns:maptor="clr-namespace:IRI.Maptor.Jab.Controls.View;assembly=IRI.Maptor.Jab.Controls">  
    . . .  
</Window>
```

2. Add the **MapView** control along with any other necessary UI elements.



```
<Grid>  
    <maptor:MapView x:Name="map" BorderBrush="Black" BorderThickness="1"/>  
    <maptor:MapInfoView Grid.Row="1" DataContext="{Binding }"/>  
    <maptor:CoordinatePanelView HorizontalAlignment="Right" VerticalAlignment="Bottom"  
        DataContext="{Binding CoordinatePanel}" Position="{Binding CurrentPoint, ElementName=map}"/>  
</Grid>
```

Configure the Map in C#

In the **code-behind**, configure the map and set up:

- **Google Maps** as the base layer
- An **initial map extent** for your view

This is done in the **Window.Loaded** event.

```
private async void Window_Loaded(object sender, RoutedEventArgs e)
{
    // initial setup
    System.Text.Encoding.RegisterProvider(CodePagesEncodingProvider.Instance);

    // Initialize map presenter (viewmodel)
    var presenter = new MapApplicationPresenter();
    await this.map.Register(presenter);
    presenter.Initialize(this);

    // Configure initial view
    presenter.SelectedMapProvider = TileMapProviderFactory.GoogleRoadMap;
    presenter.ZoomToExtent(BoundingBoxes.WebMercator_Africa, false, isNewExtent: true);
}
```

Add Core Functionalities

- **Add a Shapefile** using AddShapefileCommand



```
<Button Content="Add Shapefile" Command="{Binding AddShapefileCommand}"/>
```

- **Enable measurement tools** for length and area using the provided commands



```
<Button Content="Measure Length" Command="{Binding MeasureLengthCommand}"/>  
<Button Content="Measure Area" Command="{Binding MeasureAreaCommand}"/>
```

Full Sample Code

A complete working example is available on [GitHub](#).



Hossein Narimani Rad

